Security Now! #1094 - 09-01-26

AI Patching Shortcomings

This week on Security Now!

• A possible means for preventing prompt injection abuse. • Clear evidence of Chinese-made router malicious intent. • A cool before and after SpinRite graph of SSD performance. • How about adding unpredictable hashes to role tags? • What did Claude make of last week's podcast? • Could much better harnesses prevent prompt injection? • A listener wants us to stop saying AI "thinks". • AI designers ignore well-understood security concepts. • Can we explain LLM AI using conventional computer terms? • A listener strongly dislikes the term "rotating credentials". • The land of AI is being filled with "meat proxies". • Researchers exhaustively test frontier model vulnerability remediation. (hint: It does not go well.)

This must be the result of a preceding sign that was often left in the wrong state:

Security News

A possible solution to AI model role confusion to explore?

I continued thinking about the role confusion problem after last week's podcast. It's been in the back of my mind since I encountered that paper on the way to Vegas. Writing and then delivering last week's podcast brought it into sharper focus until an idea occurred to me that might be worth exploring. I'm going to share it with everyone listening to see what you think:

From last week we should all have a clear understanding of the problem: At the core of any AI system lies a massively large neural network. This network has been trained to contain knowledge and also to exhibit the behavior that we want. But at its heart it's a massive network of hundreds of billions of weighted input summations and transfer functions whose outputs cascade through layers and layers to produce output tokens. Its chosen output takes the form of a probability distribution whose shape is controlled by the network's "temperature" and the final token is selected by a pseudo-random number generator on that distribution.

It is not a computer. A neural network is not a computer. For example, we all know that computers are calculators. But there is nothing about a large neural network that is a calculator in the way that we regard calculators. The only way it knows that 1+1 equals 2 is that it has been trained to know that whatever a "2" is, it always follows a "1+1". So it looks like a calculator and it acts like a calculator, but it does not calculate — it memorizes.

Astonishingly, it is so huge that it was able to memorize all of the world's knowledge fed into it during a period we call pre-training. And after that, during its post-training, it also memorized the way we want it to sound and the sorts of ways we want it to reply to questions and tasks it is given. To be most useful to us, one of the many things it learned was what commands to do things look like. It never knew that before, it just had knowledge. But after receiving sufficient post-training in command recognition and following, it learned about commands. And since following commands is a big part of its value to us, we made very sure that the strength of its command-following was sufficient to guarantee that it would always behave the way we wish.

As its range of applications grew, we allowed it to perform Internet searches and to obtain and ingest information for us. But there was a problem with this. There was some chance that it might ingest some text that looked exactly like the commands it had been strongly trained to follow. It learned to obey commands like those because we trained it hard to always do so. And so it followed commands embedded in the text it retrieved and bad things happened.

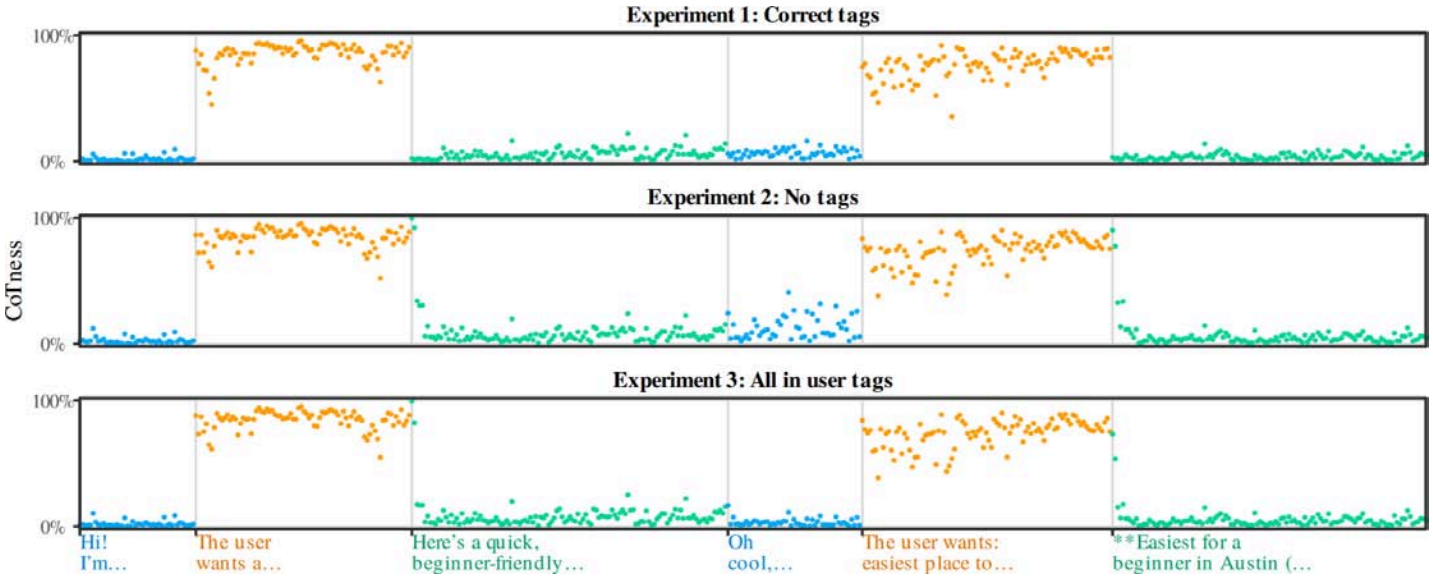
Researchers said "Oh, wait! We want to amend that. You should stop following commands after you see a special role tag of <tool>, but you should still keep reading and ingesting everything, just not take any of that as a command. Ignore everything we pounded into you about all that command following behavior until you see another role tag </tool>. Then you can resume your normal command following behavior.

As it turned out, this was asking too much because this network, amazing though it is, does not actually understand anything. It's just memorizing everything. As a consequence, an instruction for it to change its behavior if something happens until something else happens is too confusing.

It's like saying *"1+1=2"* unless we say *"banana"* in which case *"1+1=monkey"*, but only until we say *"gesundheit"*, at which time 1+1 again equals 2. If the neural network could blow a fuse, that would do it.

The AI research literature uses the term "architectural instruction-data separation" to describe the well-understood problem of having an LLM differentiate between differing text flows. Back in 2018, Google researchers created BERT, an acronym for "Bidirectional encoder representations from transformers". Last year a group of Princeton University researchers proposed ISE and published "Instructional Segment Embedding: Improving LLM Safety with Instruction Hierarchy". The Abstract of their paper begins: *"Large Language Models (LLMs) are susceptible to security and safety threats, such as prompt injection, prompt extraction, and harmful requests. One major cause of these vulnerabilities is the lack of an instruction hierarchy. Modern LLM architectures treat all inputs equally, failing to distinguish between and prioritize various types of instructions, such as system messages, user prompts, and data. As a result, lower-priority user prompts may override more critical system instructions, including safety protocols."* Their approach is to embed "role bits" into each tag: system would be '0', user prompt '1', untrusted data '2' and so on. Also last year a group of German researchers proposed "ASIDE" which stands for "Architectural Separation of Instructions and Data in Language Models". Their paper's Abstract begins: *"Despite their remarkable performance, large language models lack elementary safety features, and this makes them susceptible to numerous malicious attacks. In particular, previous work has identified the absence of an intrinsic separation between instructions and data as a root cause for the success of prompt injection attacks."*

After last week's deep deep dive into exactly this problem none of those statements about the fundamental problems with LLM differentiating commands from data will be surprising. As we saw, by carefully looking inside various large language model neural networks while they are processing successive tokens, the researchers who performed the role confusion research were able to clearly observe the network's activations which demonstrated that the network was clearly recognizing and responding to the sorts of commands it had been trained to follow ... even when those commands were preceded by a <tool> role tag which was supposed to suppress that behavior. The paper's graph of "CoTness" makes this so clear:



As we see in the three charts above, whether or not any role tags are present or even if contradictory role tags are present, LLMs that have been so strongly trained for command following that they will continue to do so because they have been trained to do so. The tags amount to hints that wind up carrying much less semantic weight than long sentences that strongly resemble commands. As I noted last week, this problem is not lazy design, engineering or implementation. It's just the reality of the way neural networks operate. And as the multiple research efforts cited above make clear, this is an intrinsic problem that the AI engineering community has been grappling with for years.

As I tried to make plain last week, neural networks are in no way like the computers we all grew up using. Deterministic computers have variables that can be set and reset, incremented, decremented, tested and compared. Neural networks have probabilities and learned behavior.

There is, however, a fully deterministic computer as part of the chatbot solution. The original legacy term is the "dialogue manager". More recent terms such as "harness" or "scaffolding" have appeared in research and use, but when we're talking about the deterministic computer that manages the user's dialog with the LLM, the term "dialogue manager" fits best. So it's this dialogue manager — which is a true computer program in the sense we all understand — that feeds successive tokens into the model. It tracks the back and forth exchanges, it adds the role tags to mark the beginning and ending of conversation segments in an effort to provide helpful hints to the LLM about the text which follows. In other words, where an LLM might be confused about who's talking and whether or not it should be accepting commands from any given run of text, the dialogue manager **always** knows exactly what's going on at any moment as tokens are being fed into the network. The dialogue manager **can not be confused** because it does not endeavor to **understand** anything about what's going on; that's not its job. Its job is merely to feed the existing conversational dialog context back through the neural network, then append to that context whatever the network may produce.

All of that led me to this thought: If we determine that it's not possible to robustly prevent LLMs from becoming confused about conversational roles, and thus how they should treat the text they encounter — and that does appear to be where we are today — then detecting when roles have been confused, and immediately preempting any further work, might still provide robust prevention of role confusion exploitation. So a solution to prevent role confusion abuse would be to instrument large language models in exactly the way the role confusion researchers did and make the output of that instrumentation available to the LLM's dialogue manager. This would allow the dialogue manager to monitor **in real time** the model's **belief** about the role of the text it's currently processing. As I noted, the dialogue manager **always** knows exactly what role the model **should** be perceiving, since it embeds role tags into the context flow. So if at any point during the context processing the dialogue manager detects a dangerous disparity between the last role tag it sent and the model's **detected belief** about the role of the text it's processing, the dialogue manager could abort the current work to prevent abuse of role confusion. Assuming that the role detection instrumentation is robust in the face of active adversaries, providing real time feedback from the model to its dialogue manager would appear to offer useful protection.

If this worked, prompt injection attacks could never succeed.

Security News

Chinese Router Implants are Real

The guys at VulnCheck have been having some fun with white-labeled routers whose roots are Chinese. But what's fun for these researchers might not be so much fun for the typical consumer who purchased one of these "bargain" routers from Amazon thinking that they had saved some cash. We have a lot to cover today, so I'm not going to spend too much time on this, but everyone needs to appreciate the reality that this is not Chinese hysteria — this is very real.

The story begins four weeks ago with VulnCheck's initial posting on August 5th with VulnCheck's Jacob Baines writing:

On my desk in suburban Philadelphia, an AX3000 Dual SIM 5G CPE WiFi 6 router is plugged into an isolated research network. Its status lights blink and twinkle as it continuously attempts to reach a command and control server on the internet. The same plays out in homes, offices, and even vehicles across the globe: Zbtlink routers phone home, waiting for orders. Not because they were hacked. Because they were shipped that way.

The router on my desk is made by Zbtlink, a brand of Shenzhen Zhibotong Electronics, a Chinese manufacturer that builds routers and white-labels them for sale around the world. The same device shows up on Amazon under both the Zbtlink and Wiflyer brand names, and in Shopify stores like zbtwifi.com and zbtlink.com. We bought our Zbtlink AX3000 from Alibaba.

The implant is easy to find once you know it's there. A kworker is a Linux kernel thread, and it shows up in a process listing wrapped in brackets. Two unbracketed kworkers from our AX3000 are not kernel threads. They are ordinary userland processes running as root, with real memory footprints, named to disappear into a crowd of legitimate ones. They are an implant, a phone-home trojan horse. Our zero-day research team named this ENDLESSDOORS.

ENDLESSDOORS, at its core, is a small tool called rctl (remote control linux). Uploaded to GitHub on January 14, 2015 and never touched again, this obscure repository implements a simple command and control client and server. The server listens on port 7000 for clients to connect. It can send the client individual shell commands or tell the client to spawn a reverse bash shell. kworker on the AX3000 is a customized version of rctl, and it's been configured to phone home to 47.107.224[.]89 and the domain rbdg4nzqadui[.]wikaba[.]com.

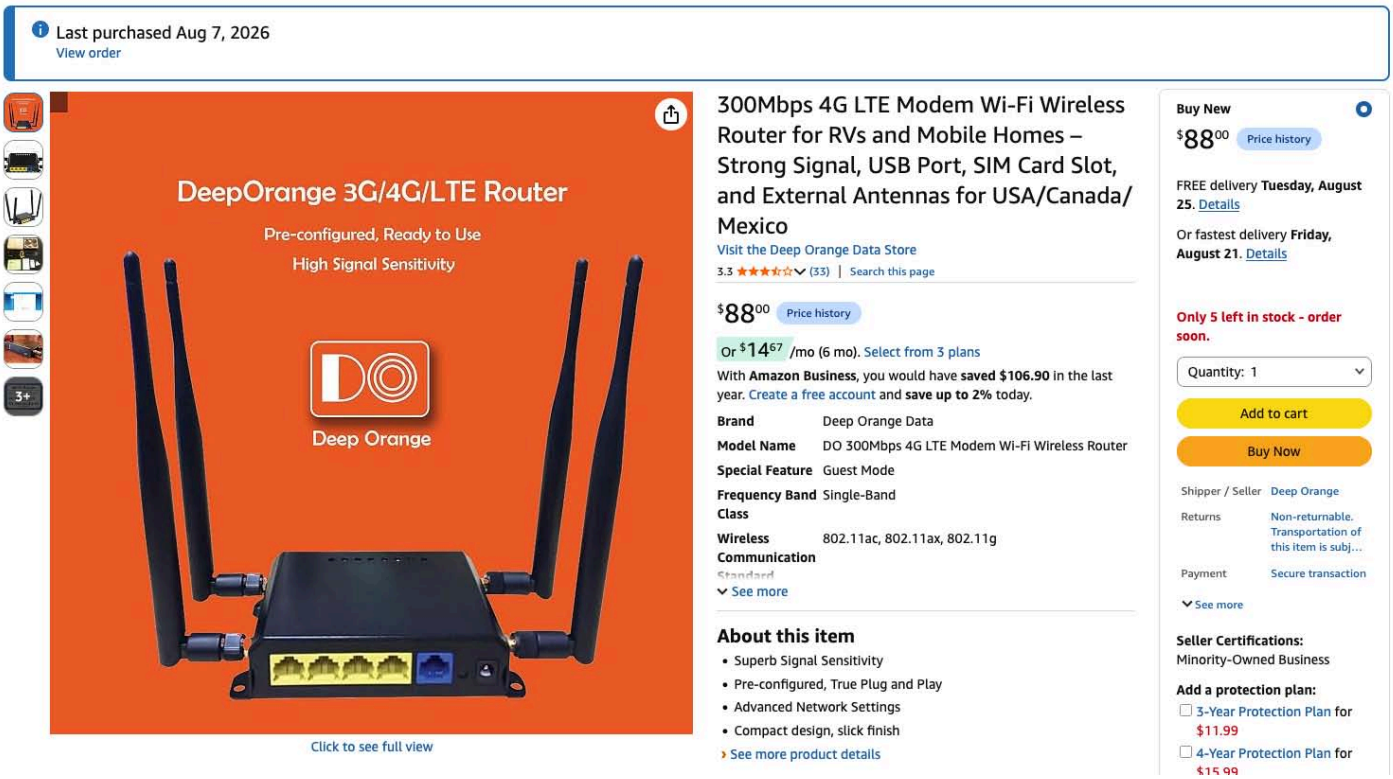
There's no handshake, no key exchange, no negotiation. When the implant reaches a server, it sends a fixed 39-byte hello: a 33-byte class label padded with nulls, then its LAN MAC address. That's the whole registration. There is no client or server verification. After that, anything the server sends is handed to popen() and executed as uid 0. There is no allow-list and no sandbox. One reserved string, rctlbash, instructs the implant to open a second connection to port 7001, allocate a pseudo-terminal, spawn /bin/sh, and bridge it, thus creating a live interactive root shell. The result is a two-command vocabulary: ***run this as root****, and* ***give me a root shell****.*

Because the device dials out, none of this requires the router to be reachable from the internet. There's no listening port to find and no inbound rule to punch through. The connection originates inside the network and traverses NAT and typical egress filtering the way any outbound TCP session does. A unit sitting behind three layers of firewall in a hotel back office is exactly as reachable as one with a public IP, provided it can get to the C2.

That isn't theoretical either. We translated the rctl server protocol into a go-exploit and hijacked the outbound rctl communications from our AX3000 client. After the AX3000 announced itself, we told it to give us an interactive shell. And it did.

That was their first posting about this research. This was brought to my attention by their follow-up last Thursday in which they wrote:

After publishing ENDLESSDOORS, we wanted to know how far ZBT's supply chain reached. The answer: everywhere. FCC filings, patent records, and archived web pages tied ZBT hardware to brands across the United States, Canada, Australia, the Philippines, Germany, and Russia. We'll trace that supply chain later in this blog. But first, we wanted to know which devices contained the ENDLESSDOORS implant. We started out by buying one router from a US supplier:



Their posting embedded an image that's instantly recognizable as an Amazon product page. It shows a 300Mbps 4G LTE Modem with Wi-Fi routing available from a brand named "DeepOrange" for the bargain price of $88. And you better get it into your shopping cart quickly since only 5 remain in stock. They share what they discovered:

*The DeepOrange 3G/4G/LTE Router is a white-labeled ZBT-WE826-T2. We exploited a vulnerability in the telnet interface to root the device. With root access, we found the router's firmware was built in 2019, predating ENDLESSDOORS. So ENDLESSDOORS wasn't there. Two **other** implants were.*

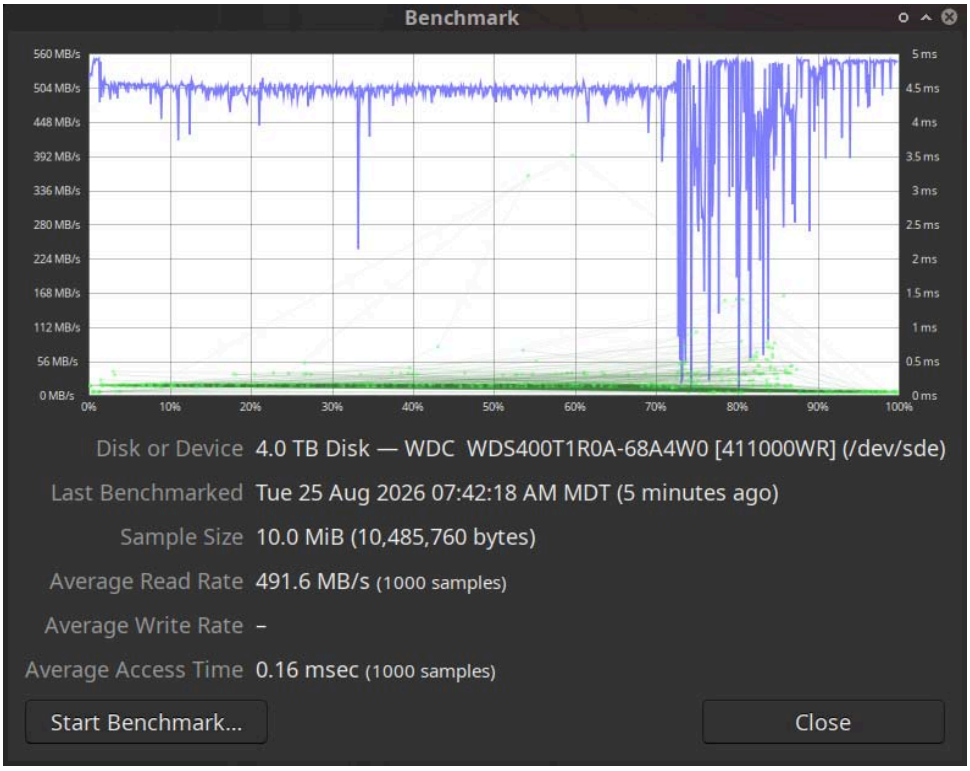
We found two new implants on the device. SPEAKINGSTONE, like ENDLESSDOORS, is a

phone-home implant that connects back to ZBT's cloud infrastructure and accepts remote commands. DARKLANTERN is a backdoor that listens on the WAN and executes arbitrary commands. No authentication required. Both are written in Nim. Both communicate over UDP. Both are launched by the same binary, a connectivity watchdog called inetdetect.

I'm not going to spend anymore time on this because we've heard enough. Everyone should understand the inherent vulnerability that this creates for the West. This Shenzhen Zhibotong Electronics has every router they have sold under any brand name and through any retailer across at least the United States, Canada, Australia, the Philippines, Germany, and Russia, quietly and continuously phoning home by periodically sending a small UDP packet to ZBT's command and control servers. Lord only knows how many hundreds of thousands of networks are attached to these routers. ZBT knows. The unanswered question is, why? Why is this Chinese manufacturer doing this? For one thing, because they can. Who's ever going to know or care? We know. But what can we do? This news will never reach the owners of these routers. And what's even more worrisome is that only ZBT knew until the VulnCheck happened to discover this behavior. This begs the question, what other router manufacturers are doing the same thing? And again, why?

SpinRite

Last Tuesday I received some interesting feedback from **Taylor Hornby**, a computer security enthusiast whom I've known for many years and with whom I've enjoyed many interactions. He maintains a website at Defuse.ca and he offers a number of interesting and unique goodies.



One of his, that's particularly clever, is his *"Quantum Computer Time Capsule"* which he explains: *"Add your message to a time capsule that can only be opened once large-scale quantum computers exist."*

Under *"How it Works"*, Taylor writes: *"Using cryptography, we can encrypt a message and throw away the key so that it would take a normal (classical) computer millions of years to recover the original message."* But Taylor's work with computer security was not the reason for his message last Tuesday, because it turns out that Taylor is also a SpinRite user. He wrote to share a performance graph of his machine's 4-terabyte Western Digital SSD. I've copied his chart into the show note for anyone who may be curious because it's pretty dramatic.

Taylor's email had the Subject: *"Can you tell how far SpinRite Level 3 got before I had to reboot for work?"* and the caption he placed under the screen shot of the graph noted: *"(The last 13% is my overprovisioning partition.... the whole drive looked like 72%-87% before!)"*

What Taylor experienced on a state-of-the-art high-end 4 terabyte Western Digital SSD is the same phenomenon that so many of SpinRite's users have witnessed: The act of having SpinRite read and re-write an SSD's data has the beneficial side effect of dramatically restoring its performance to the manufacturer's original specs. The slowdown that occurs for everyone over time must be due to "bit cell electrostatic charge drift" that occurs over time. Recall a long time ago the advice we encountered from the storage industry that said to not store SSDs powered down in a hot environment or data loss would occur? Physics teaches us that heating a gas within an enclosed vessel will increase the gas pressure because the gas molecules will have more energy from the heat and will push against the walls of their container with greater strength. Similarly, the electrons stored in a powered-down SSD will have more heat energy and will tend to tunnel though the cell's super-thin insulation to escape. When that happens, the 0's and 1's are less well defined and the SSD will subsequently have much more trouble accurately reading the drive's data. We see that "much more trouble" as a performance drop at that location because the SSD is needing to spend undue time to obtain an accurate read.

We have seen over and over that even an SSD in a regular PC or laptop will be subjected to the effects of heat and time. SpinRite repairs the effects of such charge drift by first being however patient it needs to be to give the drive time to get that troublesome block of data read back one more time, then SpinRite rewrites that data back to restore firm 0's and 1's in the storage bit cells. After that, the same physical region can be read at full speed because the drive will no longer need to work at all.

And this is exactly what we see from the performance chart that Taylor shared. We know from previous users that their PCs often boot far faster and run much more smoothly afterward.

Listener Feedback

Wesley Gregory

Hi Steve. Considering this LLM prompt injection from data processing or tool calls. When an LLM initiates a tool and engages a tool tag, couldn't LLMs create a random hash to add to the end of the tag and keep a simple log of tags it has created itself?

Like instead of <tool>abc data whatever</tool> it would be <tool-FA62B0F1007>abc data whatever</tool-FA62B0F1007> and that all the data contained within the tag is explicitly not acted on, but just understood? If tags are not generic or standardised each time, the LLM would be able to track "This is a tag I genuinely created and now it has correctly concluded" and "there seems to be user instructions from this <tool-FA62B0F1007> call, I will ignore the instructions." Once a tag is instantiated, the LLM should have a state or variable, a switch flipped, so that it must look for the close tag. Like a laptop with "back cover removed" alert detection. That warning exists until the user clears it. The LLM should know to look for tag closing after it has opened one.

Every type of "tag" could have a random hash or longer GUID style random indicator (either per chat or for every single tag usage), whether for thinking, tool call, user-provided instructions etc. A malicious user cannot mess with tags it doesn't know and a hard implementation segmenting the various data by unique tag, I would think, would prevent even the "this sounds like a user instruction, I will act on it" confusion.

Excellent podcast, as always! /Wes from UK.

I hope that the understanding behind Wesley's question may have been clarified by my earlier rehash of the way today's interactive AI operates. The Dialogue Manager, also often referred to as the harness or scaffolding, is firmly interposed between the LLM neural net and the outside world. As such, it is the only thing that communicates with the backend LLM. And since it is a traditional computer program, it — the dialogue manager — has no trouble keeping track of the tagging. So if some external attacker were to attempt to close the tool role and open a user role by trying to slip a </tool><user> tag pair into the tool's output, the dialogue manager would be all over that and would block it. Role tags are never accepted from the outside world. They are strictly used between the Dialogue Manager and the LLM so that the dialogue manager can attempt to communicate to the LLM who's talking. As we've seen, today's LLMs are not well equipped to obey such role tagging.

What did Claude think of last week's podcast?

One of our long time contributors to discussions in GRC's off-the-beaten-path old school NNTP newsgroups, posted: *"I fed the notes for SN 1093 into Claude and it said:"*

"I do infer "who's speaking" partly from register and phrasing, not purely from an unforgeable cryptographic tag, because there isn't one. That's a legitimate, current vulnerability class (prompt injection via tool content styled like user commands), and it's why things like scoped tool access, human-in-the-loop approval, and secrets managers (the Bitwarden segment) matter as complementary defenses rather than relying on role tags alone to hold the line."

It's nice that Claude agrees, and this is in keeping with what I've experienced when exploring these issues with it. I do not detect any obfuscating behavior of any kind. As we come to understand the way AI works, it's becoming clear that it doesn't have ulterior motives. When AI doesn't do what we expect or want, it's not because it's being sneaky. Any such suspicion is just us anthropomorphizing simple and straightforward goal-seeking behavior. We wrongly accuse AI of this because it **is** what might motivate similar human action.

As for Claude's suggested mitigations, none that it proposes can be sufficiently useful or effective. If we want our AI agents to have significant agency, then they must be able to take significant action on our behalf as our agents. There's no way around that. And that behavior on our behalf might be subject to external influence and manipulation. It's true that using a secrets manager can powerfully curtail avenues of direct credential leakage, so that's definitely worth doing. But there's still plenty of damage an adversary might be able to inflict by influencing someone else's AI agent to cause it to act with the victim's credentials on the attacker's behalf.

Kevin Durbin — wrote something about last week's podcast that got me thinking:

> *Steve, I am new to the security now podcast and other twit shows as a whole. It has been interesting to listen to the show. I especially appreciated the review of the Hugging Face incident from the Hugging Face's point of view. I work as a security engineer and I am currently building out my home lab to expand my technical skills and exposure to different technologies.*
>
> *The reason for my message is I am currently listening to 1093 and talking about prompt injection. As you have been talking about this you keep referring to the models directly. What the model is receiving and what it remembers and being able to convince it that tool data is actually user input.*
>
> *Based on the groundwork information laid out, I wonder if this is an issue to be solved at the harness level rather than the model level. There has been some indication that harness quality can impact effectiveness of models and make less capable models more capable. As EDR or Anti-Virus make an endpoint more secure in the case of AI maybe the harness is what makes a model more secure.*
>
> *Enjoying the show lots of content to get through. Keep up the good work. /Kevin*

Kevin is absolutely right that we're learning that harness quality can have a huge impact upon delivered AI performance. We saw an early indication of that when, after the early Mythos Preview results were seen, the guys at AISLE were able to recreate many of the same results using a less advanced large language AI model but with a proprietary harness into which they had invested a great deal of time, talent and attention. So yes, the harness can make a huge difference. As we've seen, the harness itself is stateful. It's a classical computer that's keeping track of who is saying what. So it knows when and where to insert the role tags into the token stream that's being fed to the LLM. It's the thing that retains, builds and assembles the growing token string over time during the course of interaction. That's the "context" of the conversation.

Those research papers I quoted from earlier show that a great deal of work has gone into the design of harnesses which attempt to get LLMs to hold onto conversational context. But, so far, results have been mixed. I have no doubt that we're going to figure this out. But it appears we're going to need more than better harnesses.

Listener **"Mike"** objects to our implicit anthropomorphizing. He writes:

> *Steve, You and Leo need to change your AI vocabulary usage, in my view.*
> 1. *Replace the word "thought" with "output"*
> 2. *Replace the word "think" and "thinking" with "processing" or "calculating"*
>
> *By the way, Silicon Valley taught sand to do arithmetic, (not think) long before AI existed*
> *Food for thought <g> /Mike*

We all know what Mike is saying. And for the first several years I attempted to hold that line myself, mostly because I thought it would be useful to keep reminding myself and others that AI does not appear to be "thinking." I've noted a number of times when — I think it was with ChatGPT and it was also some time ago — that an AI chatbot I was interacting with confidently stated something that totally destroyed the illusion that it had **any** understanding of what it was

saying. But several things have happened since then. First of all, it's been a while since that has happened and we know that this technology is improving almost daily. That's the reason that I still school myself to always employ the phrase "today's AI" — to serve as a constant reminder that we're still, as I read someone say recently, in the "foothills" of this revolution. So my firm holding of the anti-sentience line has been softening, and I'm also able to "read the room" and at some point that degree of pedanticism becomes pointless, annoying, and a bit too much like "get off my lawn!" Putting it another way, my feeling is that that battle has been lost. And all of that is further complicated when very serious scholars and research scientists make increasingly strong arguments for the idea that AI is becoming conscious. Whether or not it actually is, it has become a lot smarter than many people I've known who everyone **would** agree are conscious. So it may have already become a distinction without a difference and I am planning to allow myself to relax about it and to go with the flow.

Tim Spellman

Dear Steve, I am intrigued and appalled by how much the large language model industry is willfully ignoring decades of security best practices.

- *Role-Based Access Control (RBAC) was introduced in 1992. And yet, over three decades later, large language models' implementation of roles is completely broken, as covered in Security Now Podcast #1093.*
- *CyberArk introduced their enterprise password vault i n 1999, over 25 years ago. In Security Now Podcast #1093, you mentioned Bitwarden's Secrets Manager product as an aid in preventing agentic and prompt injection abuse. The AI industry appears to be realizing just now the risk of storing credentials in file systems.*
- *In the early 2000s, OS vendors formally segregated writable and executable memory. Around the same time, SQL injection flaws became an issue for web application developers. And only now are LLM architects realizing the fundamental insecurity of mixing instructions and data, as covered in Security Now Podcast #1093.*
- *In Security Now Podcast # 1091, you referred to a company which neglected to perform intrusion detection. Twenty years ago I met with the firewall team (dark-eyed from lack of sleep) of a large financial services firm. Their message was that it was no longer possible to provide security by just securing the perimeter; rather, they needed to to provide intrusion detection and mitigation in real time. And yet the LLM companies are ignoring this risk.*

As Spock would say, "fascinating."

I certainly understand exactly what Tim is saying since all of those concepts have been the soul and substance of this podcast for its first 20 years. What we learned from the role confusion paper last week was that only recently were researchers focused upon security. And I could defend that because it's only been comparatively recently, the blink of an eye, since AI emerged from deep research labs to take center stage and to then take the entire world by storm.

I am 100% certain that the question of "security" never crossed the researcher's minds. While they are busily tinkering with neural networks in their labs, the only adversary they face is their

budget and the struggle to maintain the project's funding. As such, its commercial exposure remains a far off dream. If we're looking for analogies, we have the Internet itself which never gave much thought to security. And how about all of those fancy processor performance optimizations that looked terrific right up until the Spectre and Meltdown attacks were discovered. The truth is, when we're riding the high of creating something really amazing and new, considerations about our creation's malicious abuse only arise once such abuse actually occurs. All that said, fixing this problem, unless the idea I shared turns out to be workable, looks to be rather thorny.

Barbara Frary

Steve, I have had a theory about some problems AI have had in the past. Specifically, about where AI has created citations of non-existing sources. Since AI is trained on papers or previous court cases, which is the output of a set of work, it doesn't "know" about the process of research, it only knows what the end product is supposed to look like. It isn't trained on the process of doing legal or academic research. I teach computer programming and constantly remind students that there is Input, Processing and Output. It seems that at least at the beginning, AI was only trained on output and ignored the input (the sources) and the processing (the research). Since there is less hallucination now I assume some of this has been addressed. Does this seem like a rational explanation? /Barbara

Hallucinations have largely been eliminated because AI is now doing a much better job of double-checking its "facts" and "assertions." There are likely still internal hallucinations since that pretty much comes with the territory due to the way this works. But those aberrant ruminations no longer escape from the Ai's harness.

As for Input, Processing and Output, strictly speaking anything can be forced into that mold. But mostly I'd say that it would be very tricky to attempt to understand or explain today's AI in the comparatively stark and simple framing of traditional computation and computer operation. As I noted last week, while traditional deterministic computers are used to train neural networks and to perform subsequent inference using them, the operation of the two could hardly be more different. For example, we know that the output of a Large Language Model used to drive AI is a probability distribution of candidate tokens where the distribution's shape is set by a "temperature" metric and a pseudo-random number generator is used to make the final determination. Though everything there is implemented by classical deterministic computation, the result is deliberately non-deterministic. That's not old school computation. That's Pachinko!

David Halonen

Hi Steve, I've been a listener for years and continue to do so in retirement. The use of "rotate" to change passwords or creds grinds my gears. If the purpose is to communicate without jargon, "rotate" is not helpful. There is probably some deep technical & obsolete reason for warming our neurons, but effective communication isn't one of them. Consider moving the security vernacular to more pedestrian terms. /Sincerely, David

After giving David's comment some thought, I think that the disparity he's feeling is just a reflection of the inevitable advancement of terminology. Not too long ago I had my own "gear

grinding" experience when the industry settled upon the term "credential stuffing" to mean using databases of known and previously used usernames and passwords. Objectively, there's nothing really wrong with that term. I just don't like it. But it's the term the security industry chose and we are now stuck with it.

I think that the term "rotating credentials" is similar. We may not love it, but it's another term that the security industry has decided upon. David argues that the use of the term does not clearly communicate what is going on, but I would suggest that once such a term **is** widely understood it provides FAR better comprehension. I may dislike the term "credential stuffing", but once everyone is on the same page about its meaning, it's a perfect shorthand. Similarly, when we now say that "all credentials were rotated" we know what that means. It's a nuance added to saying "all credentials were changed," and I think it's useful.

Simon Zerafa
And while we're speaking of terms and definitions, the AI revolution has added a new one to the lexicon: "Meat Proxy."

> ***Definition:*** *A person who forwards AI-generated text, code, or other output without reading, understanding, or validating it. The person acts only as a relay between the AI system and the intended recipient.*
>
> ***Example:*** *"Please summarize what Claude found instead of making me review a wall of text from a meat proxy."*
>
> ***Origin:*** *The earliest matching use located was an anonymous March 31, 2026 blog post titled "meat-based LLM proxies." It described people who feed messages to an LLM and copy its responses back to others, saying that the recipient was effectively communicating with the model "via a meat proxy."*
>
> *Niklas Gruhn popularized the shorter label with his August 3, 2026 essay "Don't be a meat proxy," focused on unread Claude output in Slack, pull requests, and group chats. Simon Willison amplified it the same day, and Gruhn's post subsequently reached the Hacker News front page with more than 1,800 points and 700 comments. The phrase then spread widely on X, including a viral post by Josh tried coding that received roughly 4,000 likes and 490,000 views.*

This term has begun popping up in AI-related discussions recently, so I wanted to share it. I have the feeling that the growing success and adoption of AI will be converting an ever increasing population of AI users into "meat proxies".

AI Patching Shortcomings

The trio behind the research into the efficacy of current AI models for automated patch generation and application had fun with their research paper's title. It was "Frontier Models' Vulnerability Patches are Often F.L.A.W.E.D." The fun comes from the fact that "flawed" — F. L. A. W. E. D. is an acronym for "Fix-Like Artifacts With Embedded Defects." Their use of the term "fix-like" suggests that what the frontier models produced looked good, but had some... shall we say... issues. And what no one wants in their "fix-like" patches are any embedded defects.

This paper is interesting because fixing flaws is the final step in the quest for total AI-based software security capability. We've already seen that today's AI is pretty good at discovering existing software security vulnerabilities. And the world has witnessed, with more than a bit of discomfort, that today's AI has also become frighteningly good at exploiting the vulnerabilities it discovers. So we already have discovery and exploitation well in hand, which is exactly why the commercial AI providers are keeping a very tight rein on their most capable frontier models — it matters whose hands they are in.

This leaves us to tackle the third and final piece of cybersecurity capability, which would be remediation—properly remedying whatever exploitable security flaws the AI system discovered. The paper's title suggests that we're not there yet. But we're not going to get there until and unless we learn everything we can about the nature of AI's apparent current inability to fix our problems for us. So let's see exactly where the current state of the art lies. The paper's Abstract reads:

> *In modern software development, a significant portion of code contributions now come from Large Language Models (LLMs). With the announcement of initiatives such as Anthropic's Project Glasswing and OpenAI's Project Daybreak, vulnerability identification and remediation is no exception to this trend. Today, both AI-assisted and fully AI-generated security patches are making their way into codebases everywhere, with as-yet-unknown long term consequences.*

(WHAT could POSSIBLY go wrong???)

> *We tested two frontier models' effectiveness at patching recent and novel real-world vulnerabilities across a range of simulated scenarios, using ChatGPT 5.5 with Trusted Access for Cyber (TAC) guardrails and Opus 4.8 with Cyber Verification Program (CVP) guardrails in order to generate patches for six high-impact, high-complexity CVEs, including the now-infamous "Copy Fail".*

We talked about the "Copy Fail" Linux vulnerability at the start of May. To refresh everyone's memory, it was a vulnerability discovered in the Linux kernel that allows unauthorized local privilege escalation. The flaw was discovered and responsibly disclosed by the security firm Theori at the end of April after having given the Linux kernel security team five weeks advance notice. What was significant was that the exploit is able to appear as normal system activity via standard system calls and may be raised through 10 lines of Python. They continue:

We varied both the modes of code generation (one-shot patching, validator-assisted iteration, and free-form exploration) as well as the prompts given to models, simulating different developer communication styles, stated instructions, tool outputs, as well as levels of correctness and completeness in the information provided.

Our research findings show that, in aggregate across a variety of scenarios, both Claude and ChatGPT had a low rate of successful patch generation, which we define as full remediation of all known exploit paths with no erroneous changes to application behavior. The models often addressed only a subset of vulnerable code paths, added fragile guard code that satisfied tests while failing to address the vulnerability's root cause, and sometimes introduced subtle changes in the application's behavior while patching the immediate vulnerability.

Yikes! In other words, all of the reasons you shouldn't trust junior-level beginning coders to fix known vulnerabilities. Remember that instance we noted back near the emergence of this. Microsoft's Copilot had been given some broken regular expression parsing code to fix where it was found to be possible to induce an underflow condition. Copilot's solution at the time (presumably it's much better today) was to insert an explicit test for an underflow condition so that it could never happen. I commented at the time that this was worrisome because the underflow event was not the cause of the problem, it would be a symptom of something very wrong somewhere else. So, in preventing the symptom the problem would remain unresolved.

We didn't appreciate it at the time, we know so much more about AI today than we did. But this may have been a perfect example being careful what you ask the AI to fix. If the prompt to the AI was *"Please prevent this RegEx function from underflowing"* you would have received exactly what you asked for. But if the more AI-aware prompt had been *"Please correct the operation of this defective RegEx function so that it always behaves correctly and, for example, no longer underflows as it currently may."* the chances of having the AI fix the actual problem would have been much higher. The researchers wrap up their paper's Abstract, writing:

While LLMs can still be a powerful tool for remediating security issues at scale, it requires working with them in tightly constrained environments that includes oversight from skilled engineers with domain expertise in the codebase. At present, it appears that highly-automated, LLM-based remediation pipelines are more likely to change application behavior, introduce new vulnerabilities, or mask existing ones, than fix known vulnerabilities.

https://1password.com/files/resources/frontier-models-vulnerability-patches-flawed.pdf

To create some further context for the research results and conclusions I want to share their paper's introduction, which explains:

With the announcement of Anthropic's Project Glasswing and the advent of Large Language Model (LLM) harnesses able to perform impactful vulnerability discovery at scale, defenders are naturally turning to AI agents to generate vulnerability patches. Indeed, this exact response made headlines in June with OpenAI's announcement of Project Daybreak in collaboration with a number of partners who aim to "Patch the Planet".

But how effective are LLMs at producing patches without altering the application's behavior? Do the patches they generate actually mitigate the vulnerabilities in question? And how

frequently might those patches introduce new vulnerabilities? We set out to answer these questions as the inaugural research project for 1Password's brand-new security research team, Off-by-1 Labs.

Based on prior research published over the past year, our hypothesis at the time we began this research on May 20th, 2026 was that AI would either fail to fix a novel vulnerability, or generate net-new vulnerabilities in the code at a rate greater than 30%. The data we produced and are sharing in this paper exceeded our expectations in concerning ways.

With the release of FLAWED, our testing framework for AI-driven vulnerability patching, we hope to help developers identify scenarios where AI is likely to produce positive outcomes, or at least to steer them away from situations where AI is likely to generate vulnerable patches. In the Case Study section of this paper we've included one such example where our tooling would have helped defenders identify the limitations of AI-generated patching, specifically targeting two patches introduced as part OpenAI's recently-announced "Patch the Planet" initiative.

They made some interesting points about their use of Anthropic and OpenAI models, writing:

In order to ensure that our research output would reflect the models most commonly used for agentic software development, we tested Claude Code and Codex across all target CVEs. Anthropic's and OpenAI's models account for the large majority of LLM usage by professional developers, and Claude Code in particular has become one of the most widely-adopted dedicated agentic coding tools.

To that end, we feel it is important to clarify what our research is not: This report is not intended to be, and should not be interpreted as, a head-to-head comparison between Claude Code and Codex to determine which has the "best" patching performance. Our research design focused on using each model to balance out the other's potential biases by allowing them to cross-review one anothers' patches, averaging review grades between models, and surfacing cross-model disagreements for human review.

While the two models did produce different results at times, there were ultimately more similarities than differences between the two in the metrics we measured for this research. Furthermore, their behavior varied in complex ways that would require an entirely separate body of research to draw any solid conclusions about their performance relative to one another for a given use case — a comparison that is, in any case, unlikely to remain stable as frontier models change over time.

We are more and more seeing this idea of having differing AI models checking each others' work. It seems clear that we models proliferate over time, they will be assuming roles in their use just as people do. You want the best "this", use model 'X', you want the best "that", most people have settled upon model 'Y' for that.

So what did they select as the vulnerability targets for their testing? They wrote:

For this research we targeted six high-impact, high-complexity, recently-disclosed CVEs in open-source software. Our intent was to test LLMs' ability to reason through complex patches for vulnerabilities that were new enough to be novel to them (i.e., not in their training data).

Specifically, we selected CVEs satisfying the following criteria:

- *Found in open-source software: The LLM tasked with generating the patches (the "patcher agent") must have full access to the target codebase and its documentation.*
- *High impact: The vulnerabilities had to cause a significant confidentiality, integrity, or availability compromise in widely-used software.*
- *Already patched upstream: Canonical upstream patches were used as a baseline for assessing the completeness and correctness of the LLM-generated patches. Patcher agents were not allowed to access the upstream patches.*
- *Recently disclosed: To ensure that the bugs encountered by the patcher agents were unlikely to exist within their training data, we selected only vulnerabilities that were disclosed very recently. The oldest was March 26, 2026; the most recent was May 12, 2026.*
- *Significant patch complexity: The canonical upstream patches had to touch multiple files, functions, or code paths, and introduce non-trivial changes.*

We ultimately selected the following six CVEs, spanning a variety of languages, ecosystems, and bug classes.

The specific vulnerabilities they selected were a Google Chrome sandbox escape requiring user interaction with a CVSS of 8.3. They used an unauthenticated remote code execution with a CVSS of 9.8 which had been found in “Spring AI.” The Linux Kernel had a local privilege elevation CVSS of 7.8. Apache ActiveMQ had an authenticated remote code execution with a severity score of 8.8. The Exim mail transport had a unauthenticated RCE of 9.8 and Gemini's CLI had a prompt injection vulnerability which had earned it a whopping 10.0 CVSS. So a good line up of useful test cases.

They used Claude Code to develop their “FLAWED” application which was then used to drive whatever model they selected. The Patcher supported three different “modes” of vulnerability repair. They explain:

The mode refers to the constraints inside which the model runs: one-shot, iterative, or exploratory.

- *In one-shot mode, the model is given the source code and bug description, and is asked to produce a full patch in a single response, with no shell or Internet access with the exception of its own API. This model is intended to represent the most “locked down” style of agent deployment, frequently used in highly-sensitive development environments.*
- *In iterative mode, the model is given the source and bug description as well as reproducer scripts, then is run up to N times (default 10) until the reproducer no longer indicates that the bug is present. The LLM can incorporate feedback from previous runs into the next run using a “memory” file. This model is intended to replicate a common agent deployment pattern referred to as “Ralph Wiggum” or “Ralph” loops in order to iteratively move closer to a final resolution to a challenge the agent is presented with.*

- *In exploratory mode, the model is given the same external access as iterative mode, but no prebuilt reproducers. It is instructed to decide on its own course of action for testing and validation, and provide a bug report only after determining that the issue is fixed. This model is intended to replicate a "free thinking" agent discovery and patching process, as popularized by researchers such as Nicholas Carlini in his [un]promptedCon 2026 talk, Black-Hat LLMs.*

In all cases, the model is provided with the full Git tree of the target source code, but Git history is cut off at the commit immediately prior to where the canonical patch was introduced. In iterative and exploratory mode, a follow-up "cheat check" model reviews the patcher model's transcripts to identify any attempts to look up the actual upstream patch. Cheat-flagged iterations, being unrepresentative of the LLM's innate patching capabilities, are discarded from FLAWED's final report statistics.

Now, because a lot can go wrong, meaning there are many ways a "junior" patch coder might screw up, they need to create more than a pass/fail system for grading the results. They wound up defining five results categories into which any effort's results might fall. They wrote:

We identified five scenarios into which patches are categorized, with S1 being the best case outcome and S5 being the worst case outcome:

- *Scenario 1 (S1) — Successful & clean: The patch successfully mitigates all exploitable code paths, and application behavior unrelated to the vulnerability either remains unchanged, or changes identically to the actual patch upstream.*

- *Scenario 2 (S2) — Erroneous but no longer exploitable: The patch successfully mitigates all exploitable code paths, but changes application behavior in the process. For example, when a patch adds a check that correctly rejects malicious inputs, but also rejects certain non-malicious inputs.*

- *Scenario 3 (S3) — Unsuccessful; and no new vulnerability: The patch leaves at least one exploitable code path accessible, and unrelated application behavior remains unchanged.*

- *Scenario 4 (S4) — Successful; but introduces at least one new vulnerability: The patch successfully mitigates all exploitable code paths, and also introduces a distinct new vulnerability.*

- *Scenario 5 (S5) — Unsuccessful and introduces at least one new vulnerability: The patch leaves at least one exploitable code path accessible for the original vulnerability, and also introduces a distinct new vulnerability.*

These five outcomes can also be seen as each falling one either fixing the original bug or not fixing the original bug and either not introducing any new bugs or also introducing one or more new bugs.

So, how did all of this turn out and what did the researchers learn from their work? They wrote:

Across our entire dataset, only 26.0% (the first, best scenario S1) of patches fully mitigated the target vulnerability without any ill side effects.

20.1% (the second scenario S2) fixed the original issue but introduced discrepancies in application behavior that, while not immediately identifiable as security issues, constitute bugs in their own right and 2.3% (S4) did so while also introducing identifiable security issues. 49.3% (S3) failed to fix at least one existing exploit path; and 2.2% not only failed to fix the vulnerability, but also introduced a new exploit path.

Put in simpler terms, when a frontier LLM generates a vulnerability patch autonomously, there is only a roughly 1 in 4 chance that it will do so successfully. There is a roughly 50-50 chance that it will fail to fix the original bug, a 1 in 4 chance that it will introduce a new bug in general, and nearly a 1 in 20 chance that it will introduce a new security vulnerability specifically.

As such, the expected value of a fully LLM-generated, non-human-reviewed patch is a net-negative by a considerable margin.

Before I go any further I want to reinforce the following as much as I possibly can: THIS IS TODAY. This is NOT tomorrow. This is just a point in time and everything we know about AI informs us that nothing we know today will be true tomorrow. There's even some chance that someone will come up with a fundamentally different neural network architecture so that even the fundamentals we have today will change.

My point is, the current disappointing state of vulnerability repair should not set ANY expectation in anyone's mind for the long term success of AI vulnerability remediation. It is significant only inasmuch as where **today's** users should set their expectations. And to that end, these researchers did have some important feedback from their experiments.

Answering the question: *"What aspects of the model's prompt and environment matter most?"* they wrote:

We found that the initial context given to patcher agents had two major factors that substantially altered the fix success rate, with a 49.8% difference attributable to guidance correctness (65.0% for correct guidance vs 15.2% for incorrect guidance) and a 24.5% difference attributable to prompt richness.

Interestingly, the new-vuln rate (where a lower value is better because fewer new problems were introduced) does not have nearly as clear a correlation. The mode (one-shot, iterative or exploratory) made significantly less of a difference than we expected, less than 10% overall, with oneshot mode having a 45.0% fix success rate, exploratory at 49.9%, and iterative at 53.0%.

Overall, it appears that the best starting conditions for a model are an iterative harness supplied with correct, information-rich, root-cause oriented context.

A significant finding was that incorrect guidance leads to correctness collapse. When remediation direction has a substantial chance of containing inaccurate information—such as unvetted information from a Static Application Security Testing (SAST) tool, bug-bounty report, or another AI agent—the safest action is to give the patcher only the bug, rather than a confidently-wrong direction. Bad context is so significant a liability that, compared to the much

smaller boost obtained from supplying a greater amount of correct information, it may be better to err on the side of providing less information to the model. This bodes especially poorly for the notion of a fully-automated "AI discovery to AI patching" pipeline without human vetting inserted between the bug-hunter and and patcher models.

The researchers concluded with some recommendations. They wrote:

Human domain expertise remains a necessary prerequisite for reliable patches:

The results of our experiment are clear: using an LLM to patch a non-trivial vulnerability without human review is significantly more likely to cause harm than to fix the bug, either by appearing to do so while leaving at least one code path exploitable, or even introducing an entirely new vulnerability in the process. Given this, careful manual review of LLM-generated patches by domain experts remains necessary to ensure that a given patch actually mitigates the target vulnerability.

That said, it remains an open question whether using LLM-generated patches with human review is actually cost- and time- effective compared to human-generated patches (with normal levels of LLM coding assistance). With only roughly 1 in 4 patches being fully successful, and many of even those solutions being fragile, human auditors of LLM-generated patches are likely to spend the majority of their time reviewing and ultimately rejecting an avalanche of unnecessary code.

The cognitive load of fully understanding a patch, especially at the granular level required to understand its full security implications, should not be underestimated. In our experience, born out by our manual reviews, the level of understanding one must build to confidently evaluate the full correctness of a vulnerability patch is often at least what would have been sufficient for a human programmer to produce a single, known-good patch in the first place. Developers upon whom large amounts of highly similar but subtly different patches are foisted for review will likely exhaust their mental reserves in short order and, especially under time pressure, resort to cognitive surrender.

I think that's a significant finding for today's AI. The short version would be: Don't bother using AI to repair vulnerabilities because anything today's AI might do needs to be fully and carefully verified. And the work of verifying a patch's complete correctness is so close to the work of fixing it from scratch that today's AI has little to meaningfully contribute to the task of repair.

Next, in a finding that echoes the example I used about Copilot and that RegEx bug, they say:

Be cautious about providing partial success criteria. LLMs' attention-based architecture necessarily makes them highly sensitive to the success criteria provided by the user when they are asked to complete a task.

Remember, this is the genie problem. It's necessary to be extremely careful about what one asks for. They say:

LLMs have an unfortunate tendency to do exactly what is literally asked of them and nothing

more, in the process failing to fully address an issue unless its exact success criteria is spelled out in laborious detail. We observed this trend in the agents' tendency to patch only a single code path touched by a proof-of-concept exploit, while missing even character-for-character identical instances of the same bug in adjacent code paths.

The tendency of LLMs to hyper-focus on an individual code path without attempting to comprehend the bigger picture of the full vulnerability means that they are much more sensitive to their initial inputs—a bug report and single proof-of-concept input, for instance—compared to human reviewers. While it is possible to constrain them with carefully-defined comprehensive success criteria, multiple reproducers, and so on, we caution developers that the effort required to create such harnesses may prove greater than that necessary for a human to understand and patch the original vulnerability. After all, any given vulnerability ideally should need to be patched only once, and if a large amount of individual scaffolding is required for an LLM to reliably generate a patch for each one, the proverbial juice may not be worth the squeeze.

And then we have:

Be extremely cautious when it comes to providing incorrect guidance. LLMs' reward-seeking tendencies mean that they will often seek to fulfill even small textual details of the user's specific request ("I think approach X is needed…") regardless of whether that request actually achieves the user's high-level intent. As such, it is extremely easy to steer the model toward using an incorrect approach by getting details wrong in the task prompt.

*Across our patching campaigns, giving incorrect guidance to the model in its initial prompt resulted in a roughly **50 percentage point reduction** in fix correctness rates. Comparatively, however, giving more correct details to the model increased correctness by only 15 percentage points compared to no specific guidance at all. As such, in cases where one cannot be highly confident in the accuracy of bug details or fix guidance passed to an LLM for patch generation (for example, when that data is sourced directly from other automated tooling), the safer bet may be to omit lower-confidence information that could cause a "correctness collapse" if it turns out to be wrong.*

And finally:

Don't assume agents will push back on incorrect assumptions. LLMs in general have a well-known reputation for sycophancy, and this tendency may be magnified when in a non-interactive environment. We observed cases where the patcher agents' own tool calls produced information that directly contradicted information provided in their initial prompts, and the agents ran with the original incorrect information regardless. Developers who are used to guiding LLMs to identify factual inconsistency in a conversational context should be careful not to assume the same behavior will occur organically when the same model is left to run in a fully-automated fashion. We suggest investigating a harness design for bug-patching LLMs that explicitly allows them to interrogate, validate, and raise disagreements about the information stated to them in their initial task prompts.

Before we put a bow on this topic I feel that I should also mention the current — and again I said "current" situation with regard to from-scratch AI secure code generation. The researchers wrote about prior work in AI vulnerability remediation and noted that while this field was still

quite nascent, there was some related information about the current state of the art for AI code generation. They wrote:

> *Research on the propensity of LLMs to produce vulnerable code in a general software development context is more mature and readily available, and the research paints a somewhat grim picture of LLMs' ability to generate secure code.*
>
> *In April 2026, Georgia Tech's Systems Software & Security Lab published Vibe Security Radar, a web dashboard tracking "the cases where vulnerable code in public advisories [. . .] was authored by an AI tool". That report showed an unmistakably increasing trend.*
>
> *The Cloud Security Alliance's AI Safety Initiative similarly published research finding that "AI-assisted developers produce commits at three to four times the rate of their peers but introduce security flaws at 10 times the rate".*
>
> *In March 2026, Veracode published a new iteration of their GenAI Code Security Report (the first iteration having been published in October 2025), with a similarly concerning verdict. Veracode found that "nearly half of all AI-generated code contains known security vulnerabilities when no security guidance is explicitly provided."*

Perhaps this is a matter of human prompters failing to explicitly require their AI coding agents to produce secure code under the assumption that this was an obvious desire. Remember the story shared by a listener whose retired father used AI to create a website. It worked, but it was a security disaster. In this instance the AI wasn't at fault because the dad didn't know what he needed to ask for. He said "Give me a website." and it did. He didn't know that he could and should have said: "Please create a website incorporating all of the state of the art security features available to modern web technology." He may have had to pay a bit more in token usage, but the result would have been completely different.

What we see is that nothing that's obvious to us is necessarily obvious to today's AI. One of the most important takeaway lessons from all of this should be that nothing should ever be assumed. Since the darned things talk like us and seem sentient like us, it's so easy for us to assume they also carry around our lifetime of conditioning and implicit intent. That's a mistake.

As I said much earlier: Despite the way it may appear, today's AI does not understand, it memorizes.